AI Assignment

Ashutosh Ravi Gupta P25CS0002

1. CHOSEN REPRESENTATION

I built a lightweight symbolic system to decide whether an English statement follows from a short English context containing facts and if–then rules. The system has no learned component: a regular-expression parser converts supported sentences into logical literals and rules, and a forward-chaining reasoner derives consequences.

Each literal has the form (negated, predicate, arguments). Unary predicates represent attributes, such as young(bob), while binary predicates represent relations, such as chase(cat, dog). Negation is stored as a flag: "The dog is not cold" becomes not cold(dog). Variables begin with "?"; expressions such as someone, something, they, and it generally map to ?x. Rules contain body literals and one head literal. Conjunctive heads are split into separate rules sharing the same body.

English sentence	Representation
Bob is young.	young(bob)
The cat does not like the dog.	not like(cat, dog)
The bald eagle chases the cat.	chase(bald_eagle, cat)
If someone is smart and kind, then they are successful.	smart(?x) AND kind(?x) -> successful(?x)
All rabbits are furry.	rabbit(?x) -> furry(?x)
If the cat is big then it is red.	big(cat) -> red(cat)

My parser uses regex patterns rather than a full grammar. It handles attribute and relational statements, supported conjunctions, "if … then …" and "X if Y" rules, universal statements such as "All rabbits are furry," and adjective-list subjects. Verbs are lemmatised by stripping a trailing "s". Pronouns bind to a quantified variable when one is present; otherwise, they bind to the last named subject. Sentences that do not match a pattern are recorded as failed. If a query cannot be parsed, the system falls back to exact normalised string matching against the context.

This representation is intentionally limited. In particular, mapping every indefinite expression to the same variable ?x means that rules involving two independently quantified entities—for example, "If someone likes someone else …"—cannot be represented faithfully. Disjunction, relative clauses, many named-entity constructions, and probabilistic statements are also outside the current parser.

2. REASONING ALGORITHMS

I use forward chaining with unification over the parsed knowledge base, with negation as failure under a closed-world assumption by default.

1. **Unification.** Variables in a rule-body literal are bound to constants in known facts. Bindings accumulate across the body, and inconsistent bindings are rejected.
2. **Body matching.** Positive literals are matched against facts indexed by (negated, predicate). A negative literal succeeds if its negation is explicitly known or, in closed-world mode, if the corresponding positive fact is absent. For an unbound variable in a negative literal, the closed-world matcher tries constants already seen.
3. **Fixpoint loop.** The reasoner repeatedly applies rules until no new facts are added or 50 rounds have run. Rules with negative body literals are visited after positive-body rules within each round.
4. **Conflict handling.** A literal is not inserted if its opposite is already known.
5. **Entailment and explanation.** A positive query is True if it is derived. Under closed-world semantics, a negative query is True when the positive version is absent. Conjunctions require every literal to hold. Each derived literal stores its rule and supporting facts so that explain can print a proof tree.

A limitation of the current negation procedure is that running negative-body rules after positive rules in each round is not sufficient for sound negation-as-failure. For example, a rule may derive big(x) from an assumed not red(x) in an early round; if red(x) is derived in a later round, the previously derived big(x) remains in the knowledge base. I have not implemented dependency-aware stratified evaluation, so conclusions depending on such assumptions can persist after the assumption is invalidated.

I also provide an open-world option. In that mode, an unbound variable in a negative body literal causes the matcher to stop rather than enumerate possible bindings. Therefore, the open-world experiment reflects both the semantic change and this implementation shortcut.

3. EXPERIMENTS ON PUBLIC DATASETS

I evaluated 3,403 examples from three public datasets after normalising them to context, statement, and flag: 2,000 examples sampled from the RuleTaker dev split, 1,200 examples from the two balanced CounterBench JSON files, and 203 examples from the FOLIO validation split. I capped the RuleTaker sample at 2,000 rows and used random_state=0. Before evaluation, I ran unit checks for 13 parser sentence shapes and for reasoner behaviours including unification, closed- versus open-world handling, conflicts, delayed negation, and pronoun binding.

I map Unknown, Uncertain, Neutral, and similar non-entailment labels to False because the assignment requires a binary True/False flag. This collapses “not entailed” and “contradicted” into one class. The reported FOLIO accuracy therefore reflects this binary remapping and is not directly comparable to published FOLIO results that preserve the original three-way labels.

The parser patterns were developed while working with these benchmark families, and I did not reserve an independent held-out set. I therefore report these results as in-sample evaluation, not as a measure of generalisation to unseen language. The RuleTaker sample is the synthetic, templated dev data rather than the paraphrased natural-language subset; its perfect score demonstrates compatibility with the benchmark's template grammar, not unrestricted English reasoning.

3.1 MAIN RESULTS

Dataset	n	Accuracy	Total time	Mean/query	Query parsed	Contexts fully accepted
RuleTaker	2,000	100.00%	1.38 s	0.69 ms	100.0%	100.0%
FOLIO	203	61.58%	0.07 s	0.34 ms	79.3%	49.3%
CounterBench	1,200	50.42%	0.21 s	0.17 ms	20.8%	83.3%
Overall	3,403	80.22%	1.66 s	0.49 ms	70.8%	91.1%

The overall 80.22% is a sample-size-weighted aggregate dominated by the 2,000 RuleTaker examples. It combines one benchmark on which the system matches the templates perfectly with two benchmarks on which it does not beat trivial baselines, so it should not be read as a standalone headline measure of general reasoning performance.

I ran the evaluation in a Kaggle CPU notebook. The notebook does not record the exact CPU model or RAM, so the timings are useful as an indication of the implementation's low inference cost but are not fully hardware-reproducible.

3.2 BASELINE COMPARISON

I calculated two baselines on the same normalised examples: a majority-class classifier and an exact string-match classifier that checks whether the normalised statement appears verbatim among the context sentences.

Dataset	Reasoner	Majority baseline	String-match baseline
RuleTaker	100.00%	51.15%	60.95%
FOLIO	61.58%	64.53%	64.53%
CounterBench	50.42%	50.42%	50.42%

The reasoner substantially exceeds both baselines on RuleTaker. On FOLIO, it is 2.95 percentage points below both baselines, so this experiment does not show an advantage over trivial prediction on that split. On balanced CounterBench, the reasoner matches the 50.42% baselines and therefore performs at chance.

3.3 CLOSED-WORLD AND OPEN-WORLD COMPARISON

Dataset	Closed-world	Open-world
RuleTaker	100.00%	75.25%
FOLIO	61.58%	64.53%
CounterBench	50.42%	50.42%

The open-world setting reduces RuleTaker accuracy by 24.75 percentage points, leaves CounterBench unchanged, and raises FOLIO from 61.58% to 64.53%. The FOLIO score is exactly the majority-class and string-match baseline (64.53%). Because open-world mode rarely derives explicit negative facts, this apparent +2.95-point gain reflects the system collapsing toward the always-False majority prediction rather than improved logical reasoning. The RuleTaker decline is consistent with its closed-world construction, but it is also affected by the unbound-variable shortcut described in Section 2.

3.4 ERROR ANALYSIS

CounterBench reveals the difference between parser acceptance and semantic understanding. A causal sentence such CounterBench shows that a sentence can parse and still be misunderstood. "Trune causes Blaf" matches my subject–verb–object pattern and gets stored as a plain relation, cause(trune, blaf), which throws away the causal meaning entirely; the probabilistic lines like "Trune~Bern(0.7)" don't parse at all. So the 83.3% context-acceptance figure only says the parser didn't reject anything it says nothing about whether the representation was right. With no usable rules in the knowledge base, forward chaining never fires (zero unification attempts), and 79% of the queries fall through to string matching. The result is 50.42%, identical to both the majority and string-match baselines, so on CounterBench the reasoner adds nothing.

FOLIO is a milder version of the same problem. Only about half the contexts parse fully. Where both the query and the context parse, accuracy is 67.1%; elsewhere it sits between 55.6% and 58.3%. Coverage clearly matters, but even a clean parse doesn't guarantee the right logic. The main things the parser can't handle are relative clauses, disjunction, and multi-word named entities.

4. EXAMPLES OF REASONING TRACES

The explain function records the rule and supporting facts for derived literals. The following is an actual multi-step positive derivation from the sampled RuleTaker dataset.

4.1 RULETAKER DATASET EXAMPLE — TWO-STEP DERIVATION

Paraphrased context summary (the first two facts below condense two separate RuleTaker sentences): Alan is young and big. Bob is nice, green, cold, and blue. Eric is young, round, and big. Harry is round and blue. The context also contains the rules “If someone is blue and round then they are kind” and “If someone is kind then they are red.”

Query: Harry is red. Gold label: True. Prediction: True. Recorded proof depth: 2.

```
red(harry)   [by rule: If someone is kind then they are red.]
  kind(harry)   [by rule: If someone is blue and round then they are kind.]
    blue(harry)   [given]
    round(harry)   [given]
```

This trace shows two successive rule applications: the facts blue(harry) and round(harry) derive kind(harry), which then derives red(harry).

4.2 RULETAKER ACCURACY BY PROOF CATEGORY

I also grouped all 2,000 RuleTaker examples using the operational proof-depth routine in the notebook. Every group was 100% accurate:

Operational category	Examples	Accuracy
Positive counterpart derived — depth 0	418	100.0%
Positive counterpart derived — depth 1	261	100.0%
Positive counterpart derived — depth 2	198	100.0%
Positive counterpart derived — depth 3	131	100.0%
Positive counterpart derived — depth 4	31	100.0%
Positive counterpart derived — depth 5	29	100.0%
Closed-world absence	464	100.0%
No recorded positive proof	468	100.0%
Total	2,000	100.0%

I computed these categories from the proof graph recorded by my implementation. For each query literal, I check whether its positive form appears in kb.proof; if it does, the query is assigned the maximum such support-chain depth. This also includes a negated query that is False because its positive counterpart was derived. “Closed-world absence” means every query literal is negated and none of its positive counterparts is in the knowledge base. “No derivation” contains the remaining queries with no recorded positive proof (including positive queries that were not derived). These definitions make the categories mutually exclusive and exhaustive: 1,068 queries in depth 0–5, 464 by closed-world absence, and 468 with no derivation, totalling 2,000. I used this implementation-derived depth because my dataset normalisation retains only context, statement, label, and source; it does not preserve any original RuleTaker depth metadata. These bins are therefore operational proof-graph depths, not the benchmark's official depth annotations.

Overall, my system works well on RuleTaker-style templated deduction and runs quickly, but it remains below trivial baselines on FOLIO and at chance on CounterBench. The main bottleneck is semantic coverage: regex acceptance does not ensure that a sentence has been represented with the right meaning. The current round-wise treatment of negation is also not stratified, so improving parser semantics and making negation sound are the two central technical priorities.